

Relatorio Tecnico

CPU RISC-V RV32I em Verilog

Projeto EEL580 – Arquitetura de Computadores

Sumário

1	Introducao	2
2	Premissas Adotadas	2
3	Arquitetura do Pipeline	2
3.1	IF (Instruction Fetch)	2
3.2	ID (Instruction Decode)	2
3.3	EX (Execute)	2
3.4	MEM (Memory Access)	3
3.5	WB (Write Back)	3
4	Tratamento de Hazards	3
4.1	Data Hazards (RAW)	3
4.2	Load-Use Hazard	3
4.3	Control Hazards	3
5	Sinais de Debug	4
6	Validacao	4
6.1	Testbench Automatizado	4
6.2	Simulacao no Digital	4
6.3	Resultados	4
6.4	Como Reproduzir	4

1 Introducao

Este projeto descreve a implementacao de uma CPU RISC-V de 32 bits (RV32I) com pipeline de 5 estagios, utilizando Verilog para integracao no simulador Digital, atendendo aos requisitos do documento `Projeto_CPU_p1.pdf`.

2 Premissas Adotadas

- O subconjunto de instrucoes RV32I exigido pelo PDF foi implementado: `add`, `addi`, `auipc`, `sub`, `and`, `andi`, `or`, `ori`, `xor`, `xori`, `sll`, `slli`, `srl`, `srli`, `lw`, `lui`, `sw`, `jal`, `jalr`, `beq`, `bne`.
- As memorias de instrucao e dados sao externas a CPU, expostas via interfaces separadas (`imem_*` e `dmem_*`), permitindo o uso das memorias ROM e RAM do Digital.
- O sinal `load_enable_i` congela toda a CPU durante a carga assincrona das memorias, preservando o estado interno.
- O reset e assincrono (`reset_i`), zerando PC, pipeline e banco de registradores.
- Nao ha modo supervisor (S Mode); apenas o pipeline de inteiros.
- A ALU foi implementada em Verilog, sem uso de blocos combinacionais multi-bit do Digital, conforme recomendado.

3 Arquitetura do Pipeline

A CPU foi organizada em 5 estagios: IF, ID, EX, MEM e WB.

3.1 IF (Instruction Fetch)

O `program_counter` mantem o PC atual e calcula PC+4. O endereco da instrucao e apresentado em `imem_addr_o` e a instrucao e lida de `imem_data_i`. Em caso de branch tomado ou jump, o PC e redirecionado para `target_addr_i`.

3.2 ID (Instruction Decode)

O `instruction_decoder` extrai opcode, rd, rs1, rs2, funct3, funct7 e gera o imediato sign-extended para os formatos R, I, S, B, U e J. A `control_unit` decodifica o opcode e gera os sinais de controle. O `register_file` realiza leitura assincrona de rs1 e rs2, com x0 sempre retornando zero. Um mux de internal forwarding WB→ID seleciona o dado do WB quando ha conflito write-before-read.

3.3 EX (Execute)

A `forwarding_unit` seleciona a origem dos operandos A e B da ALU: valor do banco (00), forward do MEM (01) ou forward do WB (10). Para `auipc`, a entrada A recebe o PC; para `lui`, recebe zero. A `alu` executa ADD, SUB, AND, OR, XOR, SLL, SRL

e `PASS_B`. O `branch_comparator` compara `rs1` e `rs2` para `beq` e `bne`. O endereço de destino é calculado como `PC+imm` (`branches/JAL`) ou `rs1+imm` (`JALR`, com `bit 0 = 0`).

3.4 MEM (Memory Access)

O endereço de memória de dados e o resultado da ALU. Para `sw`, o dado de `rs2` é escrito. Para `lw`, o dado é lido. A escrita é suprimida quando `load_enable_i` está ativo.

3.5 WB (Write Back)

Para `jal/jalr`, o valor escrito é `PC+4` (endereço de retorno). Para `lw`, o dado lido da memória. Para demais instruções, o resultado da ALU.

4 Tratamento de Hazards

4.1 Data Hazards (RAW)

Forwarding EX/MEM e EX/WB: A `forwarding_unit` detecta dependências entre a instrução no EX e as instruções em MEM/WB. Se houver dependência, o operando é encaminhado diretamente, evitando stall.

Internal forwarding WB→ID: Quando uma instrução no WB escreve em um registrador que uma instrução no ID está lendo no mesmo ciclo, a escrita síncrona do banco ainda não ocorreu. Um mux combinacional seleciona o dado do WB diretamente para o ID/EX register.

4.2 Load-Use Hazard

Quando um load está no EX e a instrução no ID depende do resultado, a `hazard_unit` gera stall no IF/ID e flush no ID/EX, inserindo uma bolha no pipeline.

4.3 Control Hazards

Branches são resolvidos no EX. Quando um branch é tomado ou um jump é executado, flush é aplicado no IF/ID e ID/EX, e o PC é redirecionado.

5 Sinais de Debug

Sinal	Bits	Descricao
pc_debug_o	32	PC atual (estagio IF)
instr_debug_o	32	Instrucao atual (estagio ID)
alu_result_debug_o	32	Resultado da ALU (estagio EX)
alu_carry_debug_o	1	Carry da ALU (debug do somador)
alu_overflow_debug_o	1	Overflow da ALU (debug do somador)
reg_debug_o	32	Valor do registrador selecionado por <code>reg_sel_i</code>
stage_if_pc_o	32	PC no estagio IF
stage_id_pc_o	32	PC no estagio ID
stage_ex_pc_o	32	PC no estagio EX
hazard_stall_o	1	Indicador de stall ativo
hazard_flush_o	1	Indicador de flush ativo

6 Validacao

6.1 Testbench Automatizado

O arquivo `tb_riscv_cpu.v` implementa um testbench com iverilog que carrega um programa de 49 instrucoes cobrindo todas as instrucoes exigidas, executa 70 ciclos e verifica 31 registradores e 2 posicoes de memoria.

6.2 Simulacao no Digital

O arquivo `test_circuit.dig` contem um circuito pronto para o simulador Digital, com ROM pre-carregada, RAM dual-port, Clock e probes de debug. Para executar: abrir o arquivo, com `Reset=1` clicar no Clock uma vez, mudar `Reset=0`, e clicar no Clock repetidamente observando os probes. Caso falha, clicar no componente da CPU e alterar o caminho do arquivo `riscv_cpu_all.v` para o local correto.

6.3 Resultados

Todos os 32 testes passaram, cobrindo: aritmetica, logica, deslocamento, memoria, controle (branch/jump), forwarding e load-use hazard.

6.4 Como Reproduzir

```
iverilog -g2012 -s tb_riscv_cpu -o tb.out \  
  tb_riscv_cpu.v alu.v branch_comparator.v \  
  control_unit.v forwarding_unit.v hazard_unit.v \  
  instruction_decoder.v pipeline_regs.v \  
  program_counter.v register_file.v riscv_cpu.v \  
vvp tb.out
```